Building Software

If you need software that is not a Python, Julia, or R package and available as a module on the cluster (current modules can be seen with the command module avail) you can request this software be compiled as a module by reaching out to orcd-help-engaging@mit.edu. If you expect there won't be many others who will be using this software, or if you need a special version of software for your job, you can compile the software yourself for personal use.

For some common software we have written recipes for how to build them on Engaging. Check the "ORCD Recipes" section on the sidebar, or check out the Howto Recipes on the Index page.

You can usually install whatever software you need in any directory you have write access to: your home, pool, or shared storage. This means the installation will be in this directory, rather than in a system-wide directory available for everyone. Most software can be installed this way, but it is not always well documented. If the binaries, or the executable files, for the software are available, you can put those in your home directory and add that path to your `$PATH` environment variable (see the Environment Variables unit in the section on the Linux Command Line for more information). Otherwise, you may have to build the software from source.

> **The `sudo` Command**
>
> You will not be able to run the `sudo` command on Engaging. The sudo command is used to run commands that could create system-wide changes that affect all users on the system. If instructions tell you to run a `sudo` command, see if you can run the command without `sudo`, or search for instructions on how to install without sudo, such as by building from source. If you are still having trouble reach out to orcd-help@mit.edu for help.

Software compilation workflows will vary based on the software and its dependencies, but this is a general workflow for compiling software. Check if the documentation has instructions for building from source and refer to those in addition to these steps.

Suggested Directory Structure

There can be a lot of moving parts when building software, and it's easy to forget where or what you installed, so it is helpful to stay organized. We recommend a directory structure that looks like this:

- $HOME/software - A directory called "software" to keep all your builds
  - [software_name] - The name of the software
    - [source_code] - The directory of source code you downloaded
    - install - For the installation files
    - deps - For any dependencies needed to build your software
      - [dep1_src] - Downloaded source for dependency 1
      - [dep2_src] - Downloaded source for dependency 2
      - install - Directory for all dependency installs so they are in one place

Step 0: Check for Pre-Built Binaries

In some cases your software will distribute pre-built binaries, or executable files, that you can run without building the software yourself. You will want to look for files that are labeled for the x86 architecture and linux operating system.

You can follow the instructions in Step 1 for downloading the binaries onto Engaging and unpacking them. Once you have done that, skip to Step X to set your `$PATH` to use the software.

Please note that this process assumes you have downloaded the source code for the software you wish to compile, and that source code is located at /home/$USER on the engaging cluster:

Step 1: Download and Unpack the Source Code

Go to your software's download or releases page and look for the source code distributions. If there are different distributions for Linux and Windows, be sure to select the one for Linux.

You can usually left-click and select "Copy Link Address" to get the URL to the download, then in a terminal on Engaging use the `wget` command to download the source. If the code is in GitHub you can check if they have tagged releases in the right column under "Releases", or clone the repository.

Often the source is distributed in `tar.gz` files, so you will need to use the `tar` command to unpack:

```
tar -xf my-software.tar.gz
```

Step 2: Start an Interactive Job for Compilation

Builds are not too computationally heavy, but tend to run more slowly on the login nodes. To start an interactive job for compilation, use the command:

```
salloc -p mit_normal -c 4
```

This will allocate 1 node from the `mit_normal` partition with 4 CPUs for compiling your software. You will receive output that your request for allocation has been submitted, and when a node has been allocated for you to use, it will say “[Node_name] are ready for job”.

Note

Be sure to request the partition you intend to run your jobs on. The node you build on will need to match the operating system of the nodes you plan to run your jobs on.

Step 3: Load Any Dependency Modules

Make sure you have a `gcc` module loaded with the `module list` command. If you don't see one, pick one of the more recent versions and load it with the `module load` command.

If your software uses `cmake` to build, you will also need to load a cmake module:

```
module load cmake
```

If your application uses MPI load an MPI module as well. Again, if you aren't sure which version to choose we recommend using a newer version.

Check your documentation for any other dependencies and load any available modules for those.

Step 4: Configure Software

The next step is often to run some kind of configure step. This is the step where you specify what build options you want or where any dependencies are installed. During this step there are usually checks for required dependencies and a working compiler. It is also at this step that you usually specify where you want your software installed.

The two most common technologies for this step are a `configure` script and `cmake`. For these your software will come with either a `configure` script or a `CMakeLists.txt` (for `cmake`). If it comes with both you can check your software's documentation to see if they recommend one over the other. You will also want to check for any additional build flags you would like enabled, or additional instructions for this step.

Running the `configure` script

First go to the directory that contains the `configure` script. When you run `configure` you will need to specify the install location, usually somewhere in your home directory. You can do this with the `--prefix` flag:

```
./configure --prefix=$HOME/[install_directory]
```

where 'install_directory' is the directory where you would like the software to be installed to. If you plan to install multiple pieces of personal software, we recommend making a folder entitled `software` in your home directory and installing software there. We like to use the path `/home/$USER/software/[software_name]/install` where `[software_name]` is the name of your software. The `install` directory is there to differentiate from any source code that may be stored in the same location. If you keep the source code elsewhere you can leave off `install` from the path.

Running `cmake`

If your software has a `CMakeLists.txt` file, it uses cmake to build and you can use the flag `-DCMAKE_INSTALL_PREFIX` to specify an install location in your home directory. If you've installed any additional dependencies you can specify their location with the `-DCMAKE_PREFIX_PATH` flag. Consult the install documentation for any additional flags for other options that you might want.

First go to the top level source code directory that should have the `CMakeLists.txt` file. Create a build directory and enter it:

```
mkdir build
cd build
```

This `build` directory is where you will run cmake.

Then use the cmake command with the option `-DCMAKE_INSTALL_PREFIX` pointing to the install location you want and `-DCMAKE_PREFIX_PATH` pointing to the location of any dependencies (if you have them). The ".." at the end tells cmake to look in the directory above for the `CMakeLists.txt` file:

```
cmake -DCMAKE_INSTALL_PREFIX=$HOME/[install_directory] -
DCMAKE_PREFIX_PATH=$HOME/path/to/deps ..
```

Step 5: Build the Software with `make`

Next you will run the 'make' command to build the software. If you have started an interactive job, you will want to specify that you want to use the CPUs you had allocated for compilation (4 in this example). You can do this by using the command:

```
make -j 4
```

This step can take a long time depending on the size of the software you are building.

If you ran the configure step with `cmake`, sometimes `cmake` will run this step for you. You will get a message that there is nothing to do.

> **Tip**
>
> One nice thing about `make` is if it gets interrupted it will pick up where it left off. If you do happen to get logged out while running `make`, make sure you have set up your environment in the same way you did the first time. This includes loading modules or setting environment variables.

Step 6: Install Software

Once your make command finishes successfully, it is time to install the software. This is done simply with the command:

```
make install
```

This command copies all the installation files, the files needed to run the software, to its final install location that you set in Step 4. This command will often fail if you don't set an install location, as the most common default is to install in a system-wide directory.

Step 7: Confirm Software Installation Was Successful

At this point, you should be able to run the software you have compiled if compiled successfully. Binaries, or the executable files, are usually placed in the `bin` directory of your install directory. You run a binary to test that it works by typing out the path to that binary. For example:

```
$HOME/software/[software_name]/install/bin/my_cmd
```

Note that this path will change depending on where you installed your software.

You may need to load some or all of the modules that you had loaded when you built the software, and you may need to set additional environment variables. Your software's documentation may help.

Step 8: Add the Software to Your PATH

The `$PATH` environment variable lists the directories that Linux searches for command executable files. While you can type out the full path to your newly installed binary as in Step 7, it can be more convenient to type only the command itself. To do that you can add the install location to your `$PATH`. It is important to set it properly, if you remove existing entries from your `$PATH` it can affect your ability to run basic commands like `ls`.

```
export PATH=$HOME/software/[software_name]/install/bin:$PATH
```

Note that this path will change depending on where you installed your software.

Examples

The Recipes section in the documentation contains several examples of builds that have been done on Engaging. Some examples include:

- Vasp
- Gromacs
- Orca (Installing pre-built binaries)

February 11, 2026